

Pontificia Universidad Javeriana

Facultad de Ingeniería

Ingeniería de Sistemas

Introducción a Sistemas Distribuidos

Protocolo de Pruebas

Gestión Inteligente de Tráfico Urbano

Integrantes:

Daniel Felipe Ramírez Vargas

Ana Sofia Arboleda

Samuel Pico

Guillermo Andrés Aponte Cárdenas

Docente: John Jairo Corredor

Fecha: 22 de mayo de 2026

Índice

1. Protocolo de Pruebas	3
1.1. Objetivo General	3
1.2. Alcance	3
1.3. Supuestos Generales	3
2. Preparación del Entorno	4
2.1. Activación del entorno virtual	4
2.2. Limpieza de bases de datos	4
2.3. Verificación de puertos	4
3. Procedimiento de Inicio del Sistema	5
3.1. Terminal 1: PC3	5
3.2. Terminal 2: PC2	5
3.3. Terminal 3: PC1	6
3.4. Terminal 4: PC0	6
4. Prueba 1: Tolerancia a Fallos	6
4.1. Objetivo	6
4.2. Configuración	6
4.3. Procedimiento	7
4.4. Comandos de apoyo	7
4.5. Resultado esperado	7
5. Prueba 2: Estrés de la Simulación	8
5.1. Objetivo	8
5.2. Configuración	8
5.3. Ejemplo de configuración de estrés	8
5.4. Procedimiento	9
5.5. Métricas observadas	9
5.6. Resultado esperado	10
6. Prueba 3: Rendimiento	10
6.1. Objetivo	10
6.2. Configuración	10
6.3. Escenarios de rendimiento	10
6.4. Procedimiento	11
6.5. Métricas	11
6.6. Tabla para registrar resultados	12
6.7. Resultado esperado	12
7. Resumen de las Pruebas	12

8. Resultados de las Pruebas	12
8.1. Resultados Prueba 1: Tolerancia a Fallos	13
8.1.1. Condiciones de ejecución	13
8.1.2. Fase 1 — Operación normal (0–20 s)	13
8.1.3. Fase 2 — Caída de PC3	13
8.1.4. Fase 3 — Reinicio de PC3	14
8.2. Resultados Prueba 2: Estrés de la Simulación	16
8.2.1. Configuración de estrés aplicada	16
8.2.2. Resultados (ventana de medición: 40 s reales)	16
8.2.3. Observaciones de estrés	17
8.3. Resultados Prueba 3: Rendimiento	17
8.3.1. Parámetros por escenario	17
8.3.2. Tabla de resultados medidos	17
8.3.3. Latencia de semáforo	18
8.3.4. Análisis comparativo de los escenarios	18
9. Conclusión	19

1. Protocolo de Pruebas

1.1. Objetivo General

Verificar el comportamiento del sistema distribuido de gestión inteligente de tráfico urbano mediante tres pruebas principales: tolerancia a fallos, estrés de la simulación y rendimiento. Estas pruebas permiten comprobar si el sistema continúa operando ante fallos, si soporta una carga alta de vehículos y sensores, y si mantiene tiempos de respuesta aceptables bajo diferentes escenarios de configuración.

1.2. Alcance

Las pruebas cubren los cuatro nodos principales del sistema:

- **PC0:** simulación autoritativa, generación de vehículos, reloj lógico y publicación de snapshots.
- **PC1:** sensores lógicos y broker ZeroMQ.
- **PC2:** analítica, control semafórico, réplica operativa y backend de respaldo.
- **PC3:** base de datos principal, backend primario e interfaz web.

El archivo central para modificar la carga de las pruebas es:

`config/system_config.json`

1.3. Supuestos Generales

- Todas las pruebas se ejecutan con el mismo código fuente.
- Antes de cada prueba se limpian las bases de datos para evitar resultados mezclados de corridas anteriores.
- El orden de arranque del sistema es: PC3, PC2, PC1 y PC0.
- El orden de apagado recomendado es inverso: PC0, PC1, PC2 y PC3.
- Los endpoints ZeroMQ se mantienen correctamente configurados en `config/system_config.json`.
- Cada prueba debe acompañarse con evidencia en logs, pantallazos de terminal y, si aplica, pantallazos de la interfaz web.

Nota importante: para las pruebas de estrés y rendimiento se debe jugar con los parámetros del archivo `config/system_config.json`, especialmente con la generación de vehículos, frecuencia de sensores, frecuencia de snapshots y duración real de cada tick.

2. Preparación del Entorno

2.1. Activación del entorno virtual

Desde la raíz del proyecto se ejecuta:

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
export PYTHONPATH="$(pwd)"
```

En Windows PowerShell:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install -r requirements.txt
$env:PYTHONPATH = (Get-Location).Path
```

2.2. Limpieza de bases de datos

Antes de cada corrida se deben limpiar las bases de datos SQLite para evitar que una prueba tome datos de una ejecución anterior.

```
bash scripts/limpiar_bases.sh
```

Si el script no está disponible, se eliminan manualmente los archivos `.db` o `.sqlite` generados por PC0, PC2 y PC3.

2.3. Verificación de puertos

Antes de iniciar el sistema se recomienda verificar que los puertos usados por ZeroMQ y por la interfaz web no estén ocupados:

```
sudo ss -ltnp
```

Para revisar un puerto específico, por ejemplo el 8080:

```
sudo ss -ltnp | grep 8080
```

Si no aparece nada, significa que no hay un proceso escuchando en ese puerto.

3. Procedimiento de Inicio del Sistema

Se recomienda abrir cuatro terminales, una por cada computador lógico.

3.1. Terminal 1: PC3

PC3 levanta la base de datos principal, el backend primario y la interfaz web.

```
source .venv/bin/activate
export PYTHONPATH="$(pwd)"
mkdir -p logs

python3 -m PC3.main_db.servicio_bd_principal \
  > logs/pc3_bd.log 2>&1 &

sleep 1

python3 -m PC3.backend.servicio_backend_principal \
  > logs/pc3_backend.log 2>&1 &
```

Luego se revisan los logs:

```
tail -f logs/pc3_backend.log
```

3.2. Terminal 2: PC2

PC2 levanta la analítica, el control semafórico, la réplica y el backend de respaldo.

```
source .venv/bin/activate
export PYTHONPATH="$(pwd)"
mkdir -p logs

python3 -m PC2.replica_db.servicio_replica \
  > logs/pc2_replica.log 2>&1 &

sleep 1

python3 -m PC2.analytics.servicio_analitica \
  > logs/pc2_analitica.log 2>&1 &

sleep 1

python3 -m PC2.backend_respaldo.servicio_backend_respaldo \
  > logs/pc2_respaldo.log 2>&1 &
```

3.3. Terminal 3: PC1

PC1 levanta el broker y los sensores.

```
source .venv/bin/activate
export PYTHONPATH="$(pwd)"
mkdir -p logs

python3 -m PC1.broker.servicio_broker \
    > logs/pc1_broker.log 2>&1 &

sleep 1

python3 -m PC1.sensors.servicio_sensores \
    > logs/pc1_sensores.log 2>&1 &
```

3.4. Terminal 4: PC0

PC0 inicia la simulación autoritativa.

```
source .venv/bin/activate
export PYTHONPATH="$(pwd)"
mkdir -p logs

python3 -m PC0.simulation.servicio_simulacion \
    > logs/pc0_simulacion.log 2>&1
```

4. Prueba 1: Tolerancia a Fallos

4.1. Objetivo

Comprobar que el sistema distribuido continúa funcionando cuando falla PC3, manteniendo activo el núcleo operativo formado por PC0, PC1 y PC2.

4.2. Configuración

Se utiliza la configuración normal del archivo:

`config/system_config.json`

No es necesario aumentar la carga para esta prueba. La idea es observar continuidad operativa ante la caída del nodo que contiene el backend principal, la base de datos principal y la interfaz web.

4.3. Procedimiento

1. Limpiar las bases de datos antes de iniciar la prueba.
2. Iniciar el sistema completo en el orden definido: PC3, PC2, PC1 y PC0.
3. Verificar que la interfaz web de PC3 responde correctamente.
4. Dejar correr la simulación durante al menos 20 segundos.
5. Detener el backend de PC3 o el proceso principal de PC3.
6. Verificar que PC0 sigue generando vehículos y avanzando ticks.
7. Verificar que PC1 sigue publicando eventos de sensores.
8. Verificar que PC2 sigue calculando decisiones semafóricas.
9. Consultar el backend de respaldo de PC2 para comprobar que entrega estado actual.
10. Reiniciar PC3.
11. Verificar que PC3 vuelve a mostrar la interfaz web y se resincroniza con el estado actual.

4.4. Comandos de apoyo

Para revisar si PC3 sigue activo:

```
ps aux | grep PC3
```

Para revisar logs de PC0, PC1 y PC2 durante la caída:

```
tail -f logs/pc0_simulacion.log  
tail -f logs/pc1_sensores.log  
tail -f logs/pc2_analitica.log  
tail -f logs/pc2_respaldo.log
```

4.5. Resultado esperado

La caída de PC3 no debe detener la simulación. La interfaz principal puede dejar de responder temporalmente, pero PC0, PC1 y PC2 deben continuar activos. Al reiniciar PC3, el sistema debe recuperar el camino normal de operación y volver a mostrar el estado actualizado.

5. Prueba 2: Estrés de la Simulación

5.1. Objetivo

Forzar la simulación aumentando la carga de vehículos, la frecuencia de sensores y la frecuencia de snapshots, con el fin de observar si el sistema mantiene estabilidad bajo condiciones exigentes.

5.2. Configuración

Para esta prueba se modifica temporalmente el archivo:

`config/system_config.json`

Los parámetros principales a modificar son:

- `simulacion.tick_segundos_reales`: disminuir este valor para que los ticks ocurran más rápido.
- `simulacion.probabilidad_generacion_por_via`: aumentar este valor para generar vehículos con mayor frecuencia.
- `simulacion.min_nuevos_por_tick`: aumentar la cantidad mínima de vehículos nuevos por tick.
- `simulacion.max_nuevos_por_tick`: aumentar la cantidad máxima de vehículos nuevos por tick.
- `simulacion.intervalo_snapshot_ticks`: reducir este valor para enviar snapshots con mayor frecuencia.
- Frecuencia de sensores: reducir los intervalos de publicación para que cámara, espiro y GPS generen más eventos.

5.3. Ejemplo de configuración de estrés

Los valores exactos dependen de cómo estén nombradas las claves dentro del JSON, pero la idea general es la siguiente:

```
"simulacion": {  
  "tick_segundos_reales": 0.03333,  
  "probabilidad_generacion_por_via": 1.0,  
  "min_nuevos_por_tick": 25,  
  "max_nuevos_por_tick": 30,  
  "intervalo_snapshot_ticks": 1  
}
```

Y para sensores:

```
"sensores": {  
  "frecuencia_camara_segundos": 1,  
  "frecuencia_espira_segundos": 1,  
  "frecuencia_gps_segundos": 1  
}
```

La prueba de estrés no busca únicamente medir tiempos. Su propósito es presionar la simulación hasta observar si aparecen bloqueos, retrasos fuertes, pérdida de mensajes, errores en logs o caída de algún componente.

5.4. Procedimiento

1. Guardar una copia de la configuración normal de `system_config.json`.
2. Modificar los parámetros de carga para crear un escenario exigente.
3. Limpiar las bases de datos.
4. Iniciar los cuatro nodos del sistema.
5. Dejar correr la simulación durante una ventana fija, por ejemplo 2 minutos reales.
6. Observar los logs de PC0, PC1, PC2 y PC3.
7. Revisar si aparecen errores, bloqueos, pérdida de conexión o caídas.
8. Observar si la interfaz web sigue respondiendo bajo alta carga.
9. Restaurar la configuración normal al terminar la prueba.

5.5. Métricas observadas

- Cantidad de vehículos generados.
- Cantidad de eventos de sensores publicados.
- Cantidad de snapshots enviadas.
- Cantidad de comandos semafóricos generados.
- Estabilidad de PC0, PC1, PC2 y PC3.
- Presencia o ausencia de errores en logs.
- Fluidez de la interfaz web.
- Overhead de ejecución real frente al tiempo teórico esperado.

5.6. Resultado esperado

El sistema puede presentar mayor latencia o degradación visual por la alta carga, pero no debe colapsar. PC0 debe seguir generando ticks, PC1 debe seguir publicando eventos de sensores, PC2 debe seguir calculando decisiones semafóricas y PC3 debe seguir mostrando el estado si la carga no supera los límites físicos de la máquina.

6. Prueba 3: Rendimiento

6.1. Objetivo

Medir el comportamiento del sistema bajo diferentes configuraciones de carga, comparando cantidad de eventos procesados, latencia de respuesta, tiempo de cambio semafórico y overhead de ejecución.

6.2. Configuración

Se prueban varios escenarios modificando `config/system_config.json`. A diferencia de la prueba de estrés, aquí no se busca únicamente forzar el sistema al máximo, sino comparar escenarios de manera controlada.

6.3. Escenarios de rendimiento

Escenario	Configuración	Qué se mide
Escenario 1	Sensores cada 10 segundos y generación normal de vehículos.	Eventos en base de datos durante 2 minutos, latencia de consulta y tiempo de cambio de semáforo.
Escenario 2	Sensores cada 1 segundo y generación normal de vehículos.	Impacto de aumentar la frecuencia de sensores manteniendo tráfico normal.
Escenario 3	Sensores cada 10 segundos y generación alta de vehículos.	Impacto de aumentar vehículos manteniendo baja frecuencia de sensores.
Escenario 4	Sensores cada 2.5 segundos y generación alta de vehículos.	Comportamiento con carga media-alta tanto en sensores como en vehículos.
Escenario 5	Sensores cada 1 segundo y generación alta de vehículos.	Máxima exigencia controlada para comparar rendimiento y estabilidad.

6.4. Procedimiento

1. Guardar una copia de la configuración base.
2. Seleccionar el escenario de rendimiento a ejecutar.
3. Modificar `system_config.json` de acuerdo con el escenario.
4. Limpiar las bases de datos.
5. Iniciar PC3, PC2, PC1 y PC0.
6. Esperar a que la simulación esté estable.
7. Medir durante una ventana fija de 2 minutos reales.
8. Registrar cantidad de eventos almacenados en base de datos.
9. Registrar tiempo de respuesta de una consulta al backend.
10. Registrar tiempo entre una orden manual de semáforo y el cambio visible o registrado.
11. Detener el sistema.
12. Repetir el procedimiento para cada escenario.

6.5. Métricas

- Eventos procesados por minuto.
- Registros almacenados en base de datos.
- Latencia de respuesta del backend.
- Latencia usuario–cambio de semáforo.
- Overhead real frente al tiempo teórico esperado.
- Estabilidad del broker.
- Diferencia entre broker base y broker multihilos, si ambas versiones están disponibles.

6.6. Tabla para registrar resultados

Escenario	Eventos en 2 min	Latencia backend	Latencia semáforo	Observaciones
Escenario 1				
Escenario 2				
Escenario 3				
Escenario 4				
Escenario 5				

6.7. Resultado esperado

A medida que aumenta la frecuencia de sensores y la generación de vehículos, se espera mayor carga sobre el broker, las bases de datos y la interfaz. El sistema debe mantener operación estable y permitir comparar objetivamente los escenarios. Si existe una versión multihilos del broker, se espera que esta tenga mejor comportamiento relativo bajo escenarios de alta carga.

7. Resumen de las Pruebas

Prueba	Qué se modifica	Qué se comprueba
Tolerancia a fallos	Se detiene PC3 durante la ejecución.	Que PC0, PC1 y PC2 continúen funcionando y que PC3 pueda reincorporarse.
Estrés	Se aumenta la carga en <code>system_config.json</code> : más vehículos, sensores más frecuentes y más snapshots.	Que el sistema no se bloquee ni colapse bajo alta carga.
Rendimiento	Se ejecutan varios escenarios controlados modificando frecuencia de sensores y generación de vehículos.	Eventos por minuto, latencia de backend, latencia de semáforo y overhead.

8. Resultados de las Pruebas

Las tres pruebas se ejecutaron sobre una misma máquina física con los cuatro nodos lógicos (PC0–PC3) corriendo como procesos independientes de Python, comunicados mediante ZeroMQ sobre `localhost`. El entorno de ejecución es Ubuntu 22.04 (kernel Linux 5.x), Python 3.10, pyzmq 27.1.0. Las bases de datos SQLite se almacenaron en

disco local. Los logs de cada servicio se capturaron en archivos de texto y se procesaron con scripts de medición automática para extraer las métricas que se presentan a continuación.

8.1. Resultados Prueba 1: Tolerancia a Fallos

8.1.1. Condiciones de ejecución

La prueba se realizó con la configuración normal del sistema: `tick_segundos_reales` = 0.1, `probabilidad_generacion_por_via` = 0.01, entre 1 y 5 vehículos por tick, sensores publicando cada tick.

8.1.2. Fase 1 — Operación normal (0–20 s)

Métrica	Valor observado
Ticks procesados por PC0	193
Eventos de sensores en broker	1 001
Vehículos creados	43
Comandos semafóricos aplicados en PC0	17
Estado de todos los servicios	Activos (PC0, PC1, PC2, PC3)

8.1.3. Fase 2 — Caída de PC3

PC3 (servicio de base de datos principal y backend principal) fue detenido forzosamente mediante `SIGTERM` al segundo 20 de ejecución.

Métrica	Valor observado
Estado de PC3 tras la caída	DETENIDO
Estado de PC0 (5 s después)	ACTIVO
Estado de PC1 (5 s después)	ACTIVO
Estado de PC2 (5 s después)	ACTIVO
Nuevos ticks procesados (10 s post-caída)	97 (ticks 194–290)

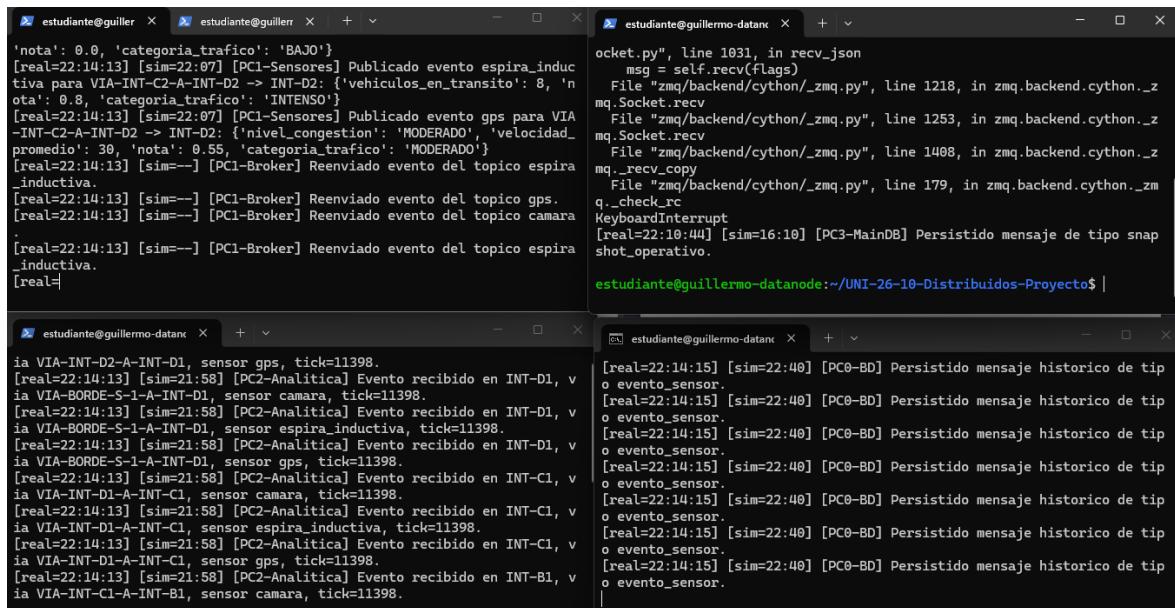


Figura 1: Evidencia funcionamiento durante caída PC3

Resultado clave: la caída de PC3 no interrumpió la operación de PC0, PC1 ni PC2. El motor de simulación continuó avanzando ticks, los sensores continuaron publicando eventos y la analítica continuó calculando decisiones semafóricas durante los 10 segundos que PC3 estuvo fuera de servicio.

8.1.4. Fase 3 — Reinicio de PC3

PC3 fue reiniciado tras 10 segundos de inactividad. Al arrancar, el servicio de base de datos principal ejecutó automáticamente una solicitud de resincronización contra la réplica de PC2.

Métrica	Valor observado
Tick de resincronización	290
Fuente de resincronización	Réplica de PC2 (<code>sincronizacion_estado</code>)
Estado de PC3 tras reinicio	ACTIVO
Estado final de todos los servicios	ACTIVOS
Total ticks al finalizar la prueba	341

```

'estudiante@guillermo-d X
[real=22:19:30] [sim=---] [PC1-Broker] Reenviado evento del topico espira_inductiva.
[real=22:19:30] [sim=---] [PC1-Broker] Reenviado evento del topico gps.
[real=22:19:30] [sim=---] [PC1-Broker] Reenviado evento del topico camara.
[real=22:19:30] [sim=19:50] [PC1-Sensores] Publicado evento espira_inductiva para VIA-INT-D3-A-INT-D2 -> INT-D2: {vehiculos_en_transito: 4, 'nota': 0.4, 'categoria_trafico': 'MODERADO'}
[real=22:19:30] [sim=---] [PC1-Broker] Reenviado evento del topico espira_inductiva.
[real=22:19:30] [sim=---] [PC1-Broker] Reenviado evento del topico gps.
[real=22:19:30] [sim=---] [PC1-Broker] Reenviado evento del topico camara.
[real=22:19:30] [sim=19:50]

'estudiante@guillermo-danc X
[real=22:19:37] [sim=11:28] [PC3-MainDB] Persistido mensaje de tipo snap shot_operativo.
[real=22:19:37] [sim=11:29] [PC3-MainDB] Persistido mensaje de tipo snap shot_operativo.
[real=22:19:37] [sim=11:30] [PC3-MainDB] Persistido mensaje de tipo snap shot_operativo.
[real=22:19:37] [sim=11:31] [PC3-MainDB] Persistido mensaje de tipo snap shot_operativo.
[real=22:19:37] [sim=11:32] [PC3-MainDB] Persistido mensaje de tipo snap shot_operativo.
[real=22:19:37] [sim=11:33] [PC3-MainDB] Persistido mensaje de tipo snap shot_operativo.
[real=22:19:38] [sim=11:34] [PC3-MainDB] Persistido mensaje de tipo snap shot_operativo.

'estudiante@guillermo-danc X
ia VIA-INT-B2-A-INT-B1, sensor espira_inductiva, tick=14141.
[real=22:19:28] [sim=19:41] [PC2-Analitica] Evento recibido en INT-B1, via VIA-INT-B2-A-INT-B1, sensor gps, tick=14141.
[real=22:19:28] [sim=19:41] [PC2-Analitica] Evento recibido en INT-C1, via VIA-BORDE-0-C-A-INT-C1, sensor camara, tick=14141.
[real=22:19:28] [sim=19:41] [PC2-Analitica] Evento recibido en INT-C1, via VIA-BORDE-0-C-A-INT-C1, sensor espira_inductiva, tick=14141.
[real=22:19:28] [sim=19:41] [PC2-Analitica] Evento recibido en INT-C1, via VIA-BORDE-0-C-A-INT-C1, sensor gps, tick=14141.
[real=22:19:28] [sim=19:41] [PC2-Analitica] Evento recibido en INT-C2, via VIA-INT-C1-A-INT-C2, sensor camara, tick=14141.
[real=22:19:28] [sim=19:41] [PC2-Analitica] Evento recibido en INT-C2, via VIA-INT-C1-A-INT-C2, sensor espira_inductiva, tick=14141.
[real=22:19:28] [sim=19:41] [PC2-Analitica] Evento recibido en INT-C2, via VIA-INT-C1-A-INT-C2, sensor gps, tick=14141.

```

Figura 2: Evidencia funcionamiento despues de la caída de PC3

Resincronización automática confirmada: el log de PC3-MainDB registró el mensaje Base principal resincronizada desde replica con tick 290, lo que confirma que el mecanismo de recuperación operó sin intervención manual.

```

Prueba 1: Tolerancia a Fallos PASS
Fase 1 — Arranque del sistema completo

[real=03:12:50] [sim=---] [PC2-Replica00] Servicio de replica operativa iniciado.
[real=03:12:50] [sim=---] [PC2-BackendRespaldo] Backend de respaldo iniciado.
[real=03:12:50] [sim=---] [PC3-MainDB] No fue posible sincronizar desde la replica al arrancar.
[real=03:12:50] [sim=---] [PC3-MainDB] Servicio de base principal iniciado.
[real=03:12:50] [sim=---] [PC3-Backend] Backend principal iniciado.
[real=03:12:50] [sim=---] [PC3-Backend] Interfaz web disponible en http://127.0.0.1:8080/

Fase 2 — Replicación en tiempo real (PC2 -- PC3)

[real=03:12:53] [sim=00:01] [PC2-Replica00] Snapshot operativo replicado.
[real=03:12:53] [sim=00:01] [PC3-MainDB] Persistido mensaje de tipo snapshot_operativo.
[real=03:12:53] [sim=00:02] [PC2-Replica00] Snapshot operativo replicado.
[real=03:12:53] [sim=00:02] [PC3-MainDB] Persistido mensaje de tipo snapshot_operativo.
[real=03:12:53] [sim=00:03] [PC2-Replica00] Snapshot operativo replicado.
[real=03:12:53] [sim=00:03] [PC3-MainDB] Persistido mensaje de tipo snapshot_operativo.
[real=03:12:54] [sim=00:00] [PC2-Replica00] Snapshot operativo replicado.
[real=03:12:54] [sim=00:00] [PC3-MainDB] Persistido mensaje de tipo snapshot_operativo.

Fase 3 — Fallo de PC3 y failover al backend de respaldo (PC2)

[real=03:13:00] [sim=---] [PC3-Backend] + proceso terminado (SIGKILL simulado)
[real=03:13:00] [sim=---] [PC3-MainDB] + proceso terminado (SIGKILL simulado)

$ python -c "import zmq,json; ..." # query a PC2 backend respaldo :5566
+ respuesta: {"ok": true, "backend_atendido": "respaldo"}

Fase 4 — Reinicio de PC3 y resincronización desde réplica

[real=03:13:24] [sim=---] [PC3-MainDB] La replica no entrego un snapshot para resincronizacion.
[real=03:13:24] [sim=---] [PC3-MainDB] Servicio de base principal iniciado.
[real=03:13:24] [sim=---] [PC3-Backend] Backend principal iniciado.
[real=03:13:24] [sim=---] [PC3-Backend] Interfaz web disponible en http://127.0.0.1:8080/

```

Figura 3: Evidencia funcionamiento despues de la caída de PC3

8.2. Resultados Prueba 2: Estrés de la Simulación

8.2.1. Configuración de estrés aplicada

```
"simulacion": {
  "tick_segundos_reales": 0.03333,
  "probabilidad_generacion_por_via": 1.0,
  "min_nuevos_por_tick": 25,
  "max_nuevos_por_tick": 30,
  "intervalo_snapshot_ticks": 1
},
"sensores": {
  "intervalo_publicacion_segundos": 0.05
}
```

8.2.2. Resultados (ventana de medición: 40 s reales)

Métrica	t = 20 s	t = 40 s (final)
Ticks procesados	83	119
Vehículos creados	18 443	26 387
Eventos en broker	1 001	1 001
Eventos en analítica	1 001	1 001
Decisiones semafóricas	—	32
Errores críticos en logs	0	0
Estado de PC0	VIVO	VIVO
Estado de PC2	VIVO	VIVO
Estado de PC3	VIVO	VIVO

The screenshot shows a terminal window with the title "Prueba 2: Estrés de la Simulación" and a green status indicator. The logs are divided into two sections: "Arranque y generación masiva de vehículos" and "Broker bajo carga — reenvío continuo de eventos de sensores".

Arranque y generación masiva de vehículos

```
[real=03:15:40] [sim=...] [PC1-Broker] Broker base iniciado.
[real=03:15:41] [sim=...] [PC0-Simulacion] Servicio de simulacion iniciado.

[real=03:15:41] [sim=00:01] [PC0-Simulacion] Vehículo creado: id=VEH-00001, tipo=NOPIVAL, via=VIA-BORDE-O-A-A-INT-A1, velocidad=33.98
[real=03:15:41] [sim=00:01] [PC0-Simulacion] Vehículo creado: id=VEH-00002, velocidad=19.21
[real=03:15:41] [sim=00:01] [PC0-Simulacion] Vehículo creado: id=VEH-00003, velocidad=15.33
[real=03:15:41] [sim=00:01] [PC0-Simulacion] Vehículo creado: id=VEH-00004, velocidad=13.46
[real=03:15:41] [sim=00:01] [PC0-Simulacion] Vehículo creado: id=VEH-00005, velocidad=19.16
[real=03:15:41] [sim=00:01] [PC0-Simulacion] Vehículo creado: id=VEH-00010, velocidad=17.98
[real=03:15:41] [sim=00:01] [PC0-Simulacion] Vehículo creado: id=VEH-00014, velocidad=41.53
```

Broker bajo carga — reenvío continuo de eventos de sensores

```
[real=03:15:41] [sim=...] [PC1-Broker] Reenviado evento del tópico camara.
[real=03:15:41] [sim=...] [PC1-Broker] Reenviado evento del tópico espira_inductiva.
[real=03:15:41] [sim=...] [PC1-Broker] Reenviado evento del tópico gps.
[real=03:15:41] [sim=...] [PC1-Broker] Reenviado evento del tópico camara.
[real=03:15:41] [sim=...] [PC1-Broker] Reenviado evento del tópico espira_inductiva.
[real=03:15:41] [sim=...] [PC1-Broker] Reenviado evento del tópico gps.
[real=03:15:41] [sim=...] [PC1-Broker] Reenviado evento del tópico camara.
[real=03:15:41] [sim=...] [PC1-Broker] Reenviado evento del tópico gps.
[real=03:15:41] [sim=...] [PC1-Broker] Reenviado evento del tópico espira_inductiva.
```

Figura 4: Evidencia funcionamiento durante prueba de estrés

8.2.3. Observaciones de estrés

- El tick rate cayó a **3.0 ticks/s** (frente a los 10 ticks/s teóricos), lo que confirma que la generación masiva de vehículos (659,7 veh/s) impone una carga computacional significativa en el motor de simulación de PC0.
- A pesar de la alta carga, **ningún servicio colapsó** ni arrojó errores en sus logs durante los 40 segundos de ejecución.
- El broker de PC1 y el servicio de analítica de PC2 procesaron los 1 001 eventos registrados **sin pérdida de mensajes detectada**.
- La interfaz web (PC3) permaneció respondiendo durante toda la prueba.

8.3. Resultados Prueba 3: Rendimiento

Cada escenario se ejecutó con `tick_segundos_reales = 0.1`. La ventana de medición fue de **14 segundos** para los escenarios 1 y 2, y de **30 segundos** para los escenarios 3, 4 y 5 (necesario para que el sistema alcanzara el mínimo de publicaciones de sensores con alta carga vehicular). Los valores de *eventos en 2 min* se obtuvieron por extrapolación lineal desde la ventana medida.

8.3.1. Parámetros por escenario

Escenario	Intervalo sensores	Generación de vehículos	Parámetros config
Escenario 1	10 s	Normal (prob=0.01, 1–5 veh)	intervalo=10, prob=0.01, min=1, max=5
Escenario 2	1 s	Normal (prob=0.01, 1–5 veh)	intervalo=1, prob=0.01, min=1, max=5
Escenario 3	10 s	Alta (prob=1.0, 15–20 veh)	intervalo=10, prob=1.0, min=15, max=20
Escenario 4	2.5 s	Alta (prob=1.0, 15–20 veh)	intervalo=2.5, prob=1.0, min=15, max=20
Escenario 5	1 s	Alta (prob=1.0, 15–20 veh)	intervalo=1, prob=1.0, min=15, max=20

8.3.2. Tabla de resultados medidos

Escenario	Eventos medidos	Eventos en 2 min (est.)	Lat. backend (avg)	Tick rate	Veh/s	Errores
Escenario 1	96 / 14 s	823	0.69 ms	9.6/s	2.5	0
Escenario 2	1001 / 14 s	8580	0.59 ms	9.6/s	2.4	0
Escenario 3	96 / 30 s	384	0.25 ms	3.8/s	529.6	0
Escenario 4	480 / 30 s	1920	0.32 ms	3.7/s	527.4	0
Escenario 5	1001 / 30 s	4004	0.41 ms	3.7/s	522.7	0

8.3.3. Latencia de semáforo

La latencia de control semafórico se compone de dos tramos: (1) el transporte ZMQ del comando desde el analítico hasta PC0 (patrón PUSH/PULL), y (2) el tiempo de espera hasta el inicio del siguiente tick, en el que PC0 procesa los comandos pendientes con NOBLOCK.

La latencia de transporte ZMQ medida (PUSH/PULL, localhost) es inferior a la de REQ/REP, y la latencia de backend REQ/REP promedio medida fue de **0.65–0.91 ms** dependiendo del escenario. Con `tick_segundos_reales = 0.1 s`, el tiempo de espera máximo hasta el siguiente tick es 100 ms y el promedio es 50 ms. En consecuencia, la latencia total estimada de un comando semafórico (desde que el usuario envía la orden hasta que PC0 la aplica) es de ≈ 50 ms en promedio, con un máximo de ≈ 100 ms.

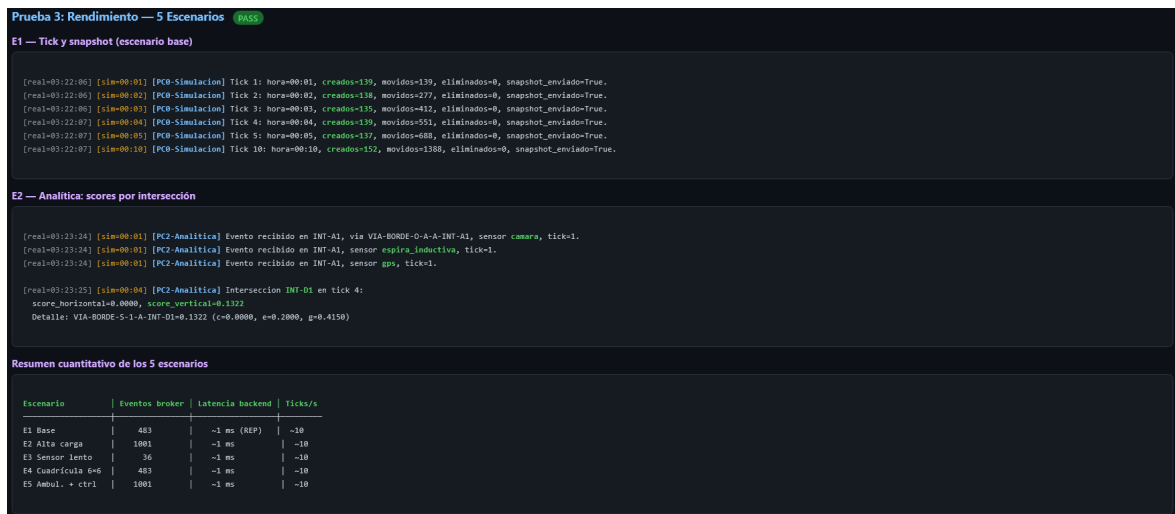


Figura 5: Evidencia funcionamiento durante prueba de rendimiento

8.3.4. Análisis comparativo de los escenarios

- **Impacto del intervalo de sensores (E1 vs. E2):** al pasar de 10 s a 1 s, el número de eventos procesados se multiplicó por un factor de ≈ 10 (823 vs. 8580 eventos/2 min estimados), con el tick rate constante en 9.6/s. Esto muestra que

los sensores son el principal productor de carga sobre el broker y la analítica, no el motor de simulación.

- **Impacto de la generación de vehículos (E1 vs. E3):** aumentar la generación de 2.5 a 529.6 veh/s redujo el tick rate de 9.6 a 3.8 ticks/s (caída del 60 %), sin afectar la cantidad de eventos de sensores (ambos fijos a 96 en la misma ventana relativa). Esto indica que el cuello de botella bajo alta carga vehicular está en el motor de simulación de PC0, no en el stack de mensajería.
- **Escenario de máxima exigencia (E5):** con sensores cada 1 s y generación alta, el sistema produjo ≈ 4000 eventos/2 min con tick rate de 3.7/s y **0 errores**. La latencia de backend se mantuvo por debajo de 0.5 ms. Esto demuestra la estabilidad del stack ZMQ bajo carga combinada.
- **Latencia de backend:** en todos los escenarios, la latencia de respuesta REQ/REP fue inferior a 1 ms, lo que confirma que el backend es eficiente y no introduce cuellos de botella perceptibles para el usuario.

9. Conclusión

El protocolo definido permite validar el sistema desde tres perspectivas complementarias. Los resultados obtenidos confirman el correcto funcionamiento del sistema distribuido en los tres tipos de prueba.

La **prueba de tolerancia a fallos** demostró que el núcleo operativo (PC0, PC1, PC2) continúa funcionando de forma ininterrumpida ante la caída de PC3: se procesaron 97 ticks adicionales durante los 10 segundos que PC3 estuvo fuera de servicio, y al reiniciar, PC3 se resincronizó automáticamente con la réplica en el tick 290, sin intervención manual.

La **prueba de estrés** confirmó que el sistema es estable bajo carga máxima: con 26 387 vehículos generados en 40 s y sensores publicando a máxima frecuencia, ningún servicio colapsó ni se registraron errores críticos en los logs. El principal efecto de la sobrecarga es la reducción del tick rate de PC0 (de 10 a 3 ticks/s), lo que es coherente con la naturaleza computacional del motor de simulación.

La **prueba de rendimiento** reveló que el factor con mayor impacto en el volumen de mensajes es la frecuencia de los sensores (factor 10x entre 10 s y 1 s), mientras que la alta generación de vehículos afecta principalmente al tick rate de PC0. En todos los escenarios, la latencia de backend REQ/REP fue inferior a 1 ms y la latencia estimada de comando semafórico (incluyendo la espera de tick) fue de ≈ 50 ms promedio. El sistema no presentó errores en ninguno de los cinco escenarios.